ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 17

Детализация Push

Рассмотрим возможности push-уведомлений, добавив в приложение заметок функционал **напоминаний**. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Введение

Ранее был рассмотрен процесс отправки push-уведомления при создании новой заметки. Однако в реальных приложениях часто требуется более гибкое управление уведомлениями. Это могут быть напоминания о событиях, откладывание на определённый срок (snooze) или персонализация сообщений.

В рамках данного занятия рассмотрим реализацию:

- Добавления заметок с указанием даты и времени напоминания.
- Планирования push-уведомлений на стороне сервера.
- Отображения кнопок в уведомлении для возможности отложить напоминание.
- Обработки действия «Отложить на 5 минут» через Service Worker.

Шаг 1. Модификация клиентской части

1.1. Добавление поля даты и времени в форму

Отредактируем файл `content/home.html`, добавив поле выбора даты и времени для напоминания. Форма будет содержать два инпута: текстовое поле для самой заметки и поле `datetime-local` для выбора времени напоминания.

```
<div class="home-content">
    <h2 class="is-center">Добавить заметку</h2>
    <form id="note-form" class="row is-center">
```

```
        <input class="col-9" type="text" id="note-input"
placeholder="Введите текст заметки" required>
        <button class="col-3 button primary" type="submit">Добавить</button>
    </form>

    <form id="reminder-form" class="row is-center" style="margin-top:
1rem;">
        <input class="col-5" type="text" id="reminder-text"
placeholder="Текст напоминания" required>
        <input class="col-4" type="datetime-local" id="reminder-time"
required>
        <button class="col-3 button success" type="submit">Добавить с
напоминанием</button>
    </form>

    <h2 class="is-center" style="margin-top: 2rem;">Список заметок</h2>
    <ul id="notes-list" style="list-style: none; padding-left: 0;"></ul>
</div>
```

1.2. Обновление логики в `app.js`

В `app.js` добавим поддержку новой формы, а также изменим структуру сохраняемых заметок, добавив поле `reminder` (timestamp напоминания) и уникальный идентификатор.

Добавим глобальную переменную `socket`, которая уже существует из предыдущей практики. Расширим функцию `initNotes()` для обработки обеих форм и отображения заметок с признаком наличия напоминания.

```
function initNotes() {
    const form = document.getElementById('note-form');
    const input = document.getElementById('note-input');
    const reminderForm = document.getElementById('reminder-form');
    const reminderText = document.getElementById('reminder-text');
    const reminderTime = document.getElementById('reminder-time');
    const list = document.getElementById('notes-list');

    function loadNotes() {
        const notes = JSON.parse(localStorage.getItem('notes') || '[]');
        list.innerHTML = notes.map(note => {
            let reminderInfo = '';
            if (note.reminder) {
                const date = new Date(note.reminder);
                reminderInfo = `<br><small>!!! Напоминание:
${date.toLocaleString()}</small>`;
            }
            return `<li class="card" style="margin-bottom: 0.5rem; padding:
0.5rem;">
```

```
                        ${note.text}${reminderInfo}
                    </li>`;
        }).join('');
    }

    function addNote(text, reminderTimestamp = null) {
        const notes = JSON.parse(localStorage.getItem('notes') || '[]');
        const newNote = { id: Date.now(), text, reminder: reminderTimestamp
};
        notes.push(newNote);
        localStorage.setItem('notes', JSON.stringify(notes));
        loadNotes();

        // Отправляем событие на сервер (только если есть напоминание)
        if (reminderTimestamp) {
            socket.emit('newReminder', {
                id: newNote.id,
                text: text,
                reminderTime: reminderTimestamp
            });
        } else {
            // Можно оставить старый эмит для уведомлений о новых заметках
            socket.emit('newTask', { text, timestamp: Date.now() });
        }
    }

    // Обработка обычной заметки
    form.addEventListener('submit', (e) => {
        e.preventDefault();
        const text = input.value.trim();
        if (text) {
            addNote(text);
            input.value = '';
        }
    });

    // Обработка заметки с напоминанием
    reminderForm.addEventListener('submit', (e) => {
        e.preventDefault();
        const text = reminderText.value.trim();
        const datetime = reminderTime.value;
        if (text && datetime) {
            const timestamp = new Date(datetime).getTime();
            if (timestamp > Date.now()) {
                addNote(text, timestamp);
                reminderText.value = '';
                reminderTime.value = '';
            } else {
                alert('Дата напоминания должна быть в будущем');
            }
```

```
        }
    });

    loadNotes();
}
```

1.3. Обновление Service Worker для обработки действий

В файле `sw.js` добавим обработчик события `notificationclick`, который будет реагировать на нажатие кнопки «Отложить».

```
self.addEventListener('notificationclick', (event) => {
    const notification = event.notification;
    const action = event.action;

    if (action === 'snooze') {
        // Получаем id напоминания из данных уведомления
        const reminderId = notification.data.reminderId;
        // Отправляем запрос на сервер для откладывания
        event.waitUntil(
            fetch(`/snooze?reminderId=${reminderId}`, { method: 'POST' })
                .then(() => notification.close())
                .catch(err => console.error('Snooze failed:', err))
        );
    } else {
        // При клике на само уведомление просто закрываем его
        notification.close();
    }
});
```

Также изменим обработчик `push`, добавив кнопку и передав `reminderId`.

```
self.addEventListener('push', (event) => {
    let data = { title: 'Новое уведомление', body: '', reminderId: null };
    if (event.data) {
        data = event.data.json();
    }
    const options = {
        body: data.body,
        icon: '/icons/favicon-128x128.png',
        badge: '/icons/favicon-48x48.png',
        data: { reminderId: data.reminderId } // для идентификации в click
    };
    // Добавляем кнопку только если это напоминание
    if (data.reminderId) {
        options.actions = [
            { action: 'snooze', title: 'Отложить на 5 минут' }
        ];
```

```
    }
    event.waitUntil(
        self.registration.showNotification(data.title, options)
    );
});
```

Шаг 2. Доработка сервера

2.1. Хранение запланированных напоминаний

На сервере (`server.js`) создадим структуру для хранения напоминаний и таймеров.

```
// Хранилище активных напоминаний: ключ - id заметки, значение - объект с
таймером и данными
const reminders = new Map();
```

Добавим обработку нового события `newReminder` от клиента:

```
io.on('connection', (socket) => {
    // ... существующий код ...

    socket.on('newReminder', (reminder) => {
        const { id, text, reminderTime } = reminder;
        const delay = reminderTime - Date.now();
        if (delay <= 0) return;

        // Сохраняем таймер
        const timeoutId = setTimeout(() => {
            // Отправляем push-уведомление всем подписанным клиентам
            const payload = JSON.stringify({
                title: '!!! Напоминание',
                body: text,
                reminderId: id
            });

            subscriptions.forEach(sub => {
                webpush.sendNotification(sub, payload).catch(err =>
console.error('Push error:', err));
            });

            // Удаляем напоминание из хранилища после отправки
            reminders.delete(id);
        }, delay);

        reminders.set(id, { timeoutId, text, reminderTime });
    });
```

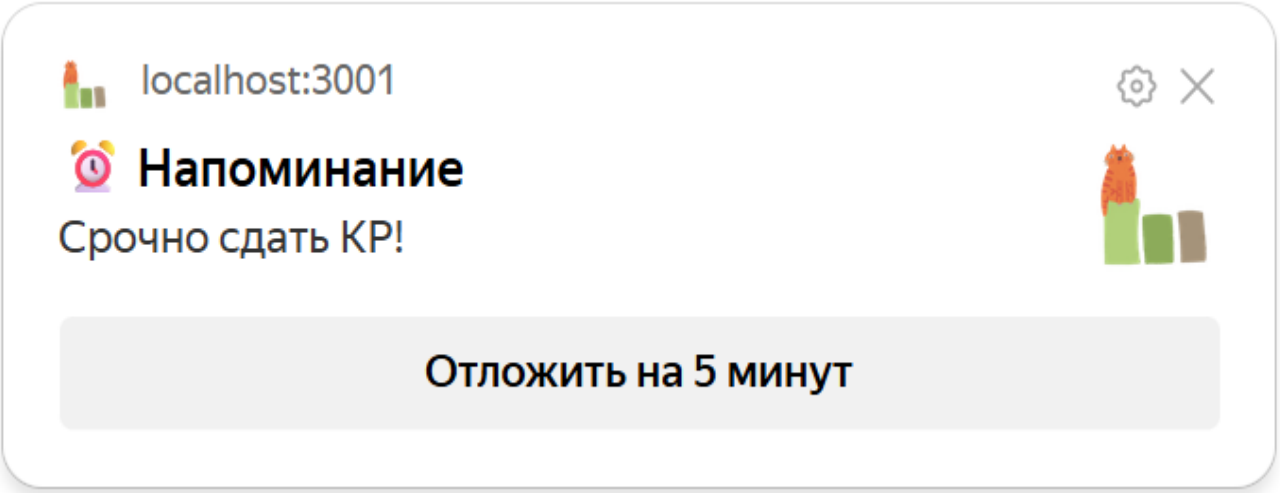
```
    // ... остальной код ...
});
```

2.2. Эндпоинт для откладывания

Добавим новый маршрут `/snooze`, который будет принимать `reminderId` и переносить напоминание на 5 минут.

```
app.post('/snooze', (req, res) => {
    const reminderId = parseInt(req.query.reminderId, 10);
    if (!reminderId || !reminders.has(reminderId)) {
        return res.status(404).json({ error: 'Reminder not found' });
    }

    const reminder = reminders.get(reminderId);
    // Отменяем предыдущий таймер
    clearTimeout(reminder.timeoutId);

    // Устанавливаем новый через 5 минут (300 000 мс)
    const newDelay = 5 * 60 * 1000;
    const newTimeoutId = setTimeout(() => {
        const payload = JSON.stringify({
            title: 'Напоминание отложено',
            body: reminder.text,
            reminderId: reminderId
        });

        subscriptions.forEach(sub => {
            webpush.sendNotification(sub, payload).catch(err =>
console.error('Push error:', err));
        });

        reminders.delete(reminderId);
    }, newDelay);

    // Обновляем хранилище
    reminders.set(reminderId, {
        timeoutId: newTimeoutId,
        text: reminder.text,
        reminderTime: Date.now() + newDelay
    });

    res.status(200).json({ message: 'Reminder snoozed for 5 minutes' });
});
```

Пример реализации откладывания представлен на Рисунке ниже



Практическое задание

Доработайте приложение из практики №16, добавив функционал напоминаний:

1. Добавьте форму для создания заметки с напоминанием (текст + дата/время).
2. Измените структуру данных в `localStorage` : заметки должны содержать уникальный идентификатор и поле `reminder` (timestamp).
3. Реализуйте на сервере планирование push-уведомлений с помощью `setTimeout` и хранение активных таймеров.
4. Добавьте в Service Worker обработку действий уведомлений: кнопку «Отложить на 5 минут» и запрос к серверу для переноса напоминания.
5. Убедитесь, что напоминания работают даже при закрытом приложении.

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой.